DOING IT MANY TIMES

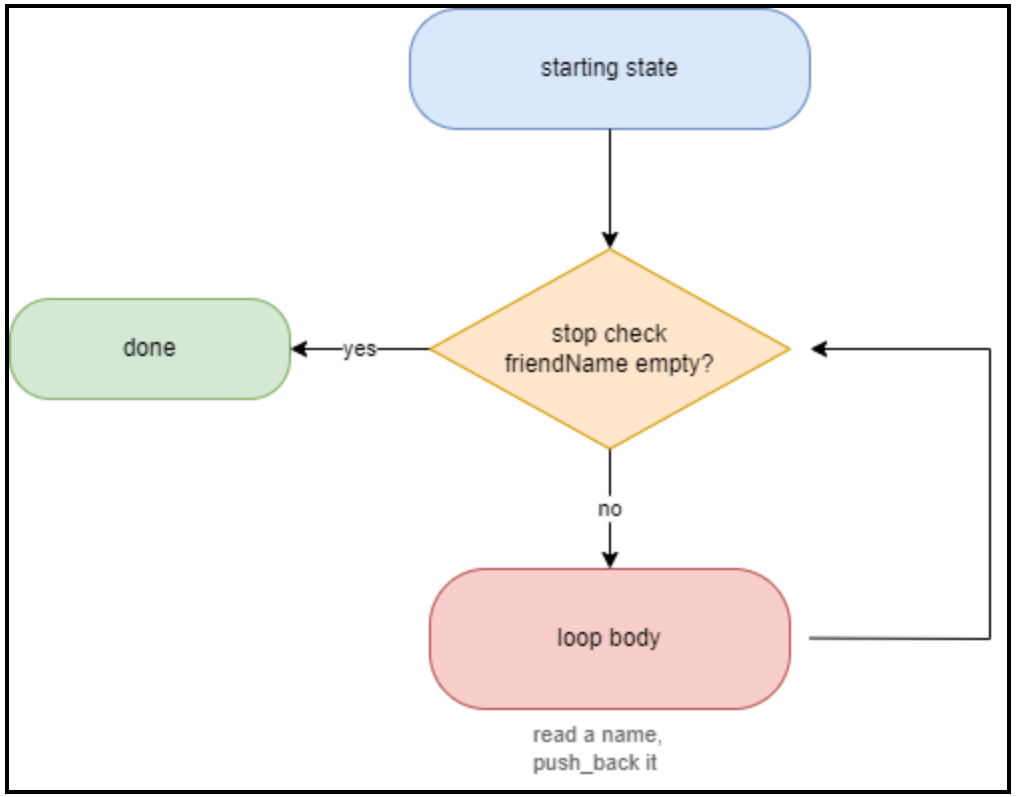
Loops and your first collection

WHAT WE ARE AFTER

Collecting a list of friends

- We tell the user to type in a friend's name.
- We keep asking until they hit enter on an empty line.
- If they typed a name, we add it to a list of friends.
- If they hit enter on an empty line, we stop asking and print the list of friends.

DISSECTING A WHILE LOOP



while - repeat until told to stop

```
while (true) {
    std::print("Friend's name: ");
    std::string friendName;
    std::getline(std::cin, friendName);

    if (friendName.empty()) {
        break; // empty line means we're done
    }

    friends.push_back(friendName);
}
```

break

`break` exits a loop immediately, regardless of the loop's condition.

```
if (friendName.empty()) {
    break;
}
```

Without it, `while (true)` would never stop on its own.

A COLLECTION, READY-MADE

You don't have to build one

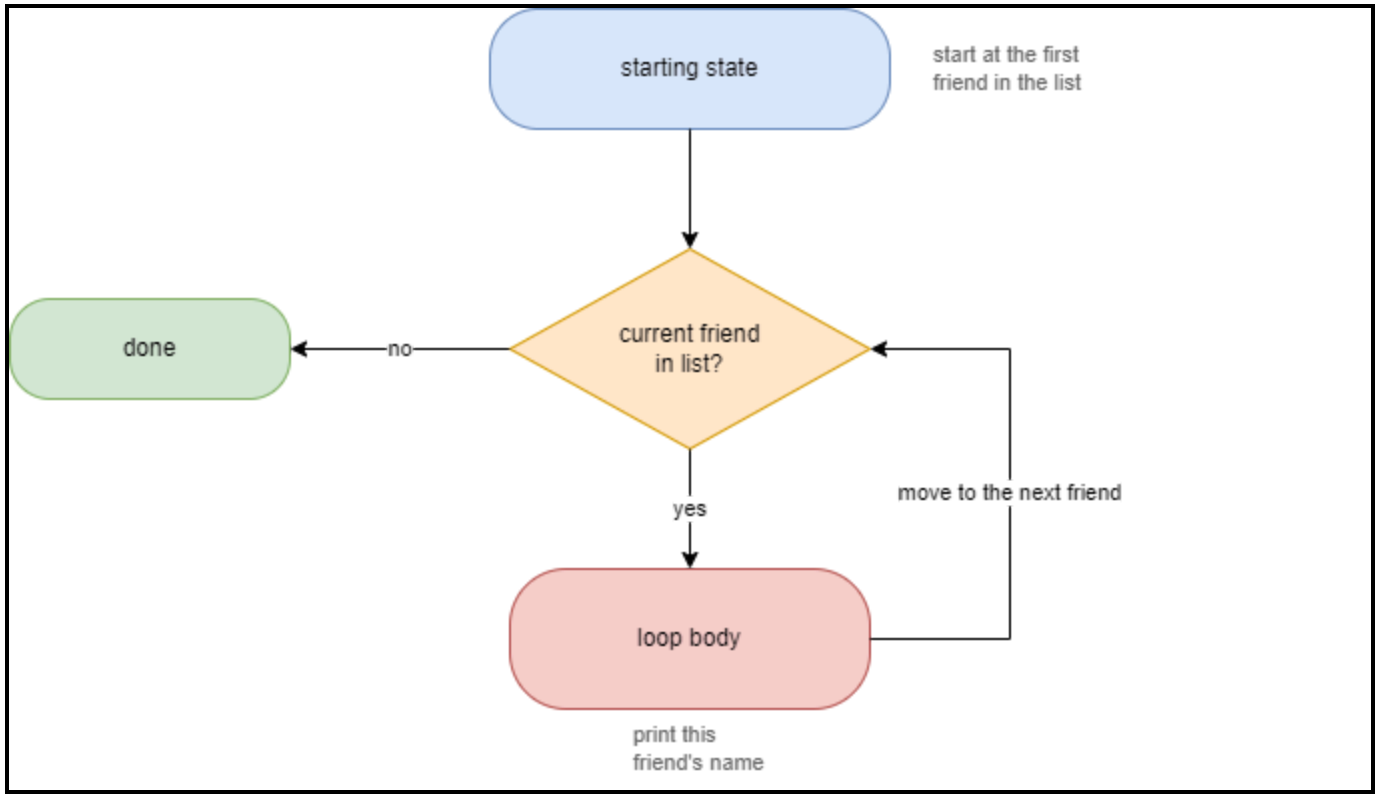
C++ ships with ready-made collections - you don't have to build your own growable list.

```
std::vector<std::string> friends;
```

`std::vector<std::string>` is a list of strings that **grows** as you add to it with `push_back`.

> Same idea as `std::string` being a ready-made collection of characters. You don't need to know how `std::vector` works inside to use one.

DISSECTING A FOR LOOP



Looping over the collection

```
std::println("\nYou added {} friend(s):", friends.size());
for (const std::string& friendName : friends) {
    std::println(" - {}", friendName);
}
```

A range-based `for` visits every element, one at a time - no manual indexing required. It's another variant of loops in C++.

DEBUG THE COLLECTION

Watch it grow, one push_back at a time

Step into the loop and open your IDE's variable/locals view - watching `friends` gain an element on each iteration is the clearest way to see a `std::vector` actually grow.